

PATRONES DE CREACIÓN BASADOS EN CLASES DEL MÉTODO FACTORY

Presentado por:

- Andrés Carrillo
- Franco Quiroz

UNIDADES TECNOLÓGICAS DE SANTANDER

Programación en Java

Pr. Ricardo Andres Medina

Bucaramanga –Santander

28-Agosto-2026



Tabla de Contenido

CONCEPTO	-----	3
EJEMPLO	-----	4
IMPLEMENTACIÓN	-----	7
PROS Y CONTRA	-----	8
RELACIÓN CON OTROS PATRONES	-----	9
EJEMPLOS DE CÓDIGOS	-----	10
REFERENCIAS	-----	11



FACTORY METHOD

Es un Patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.

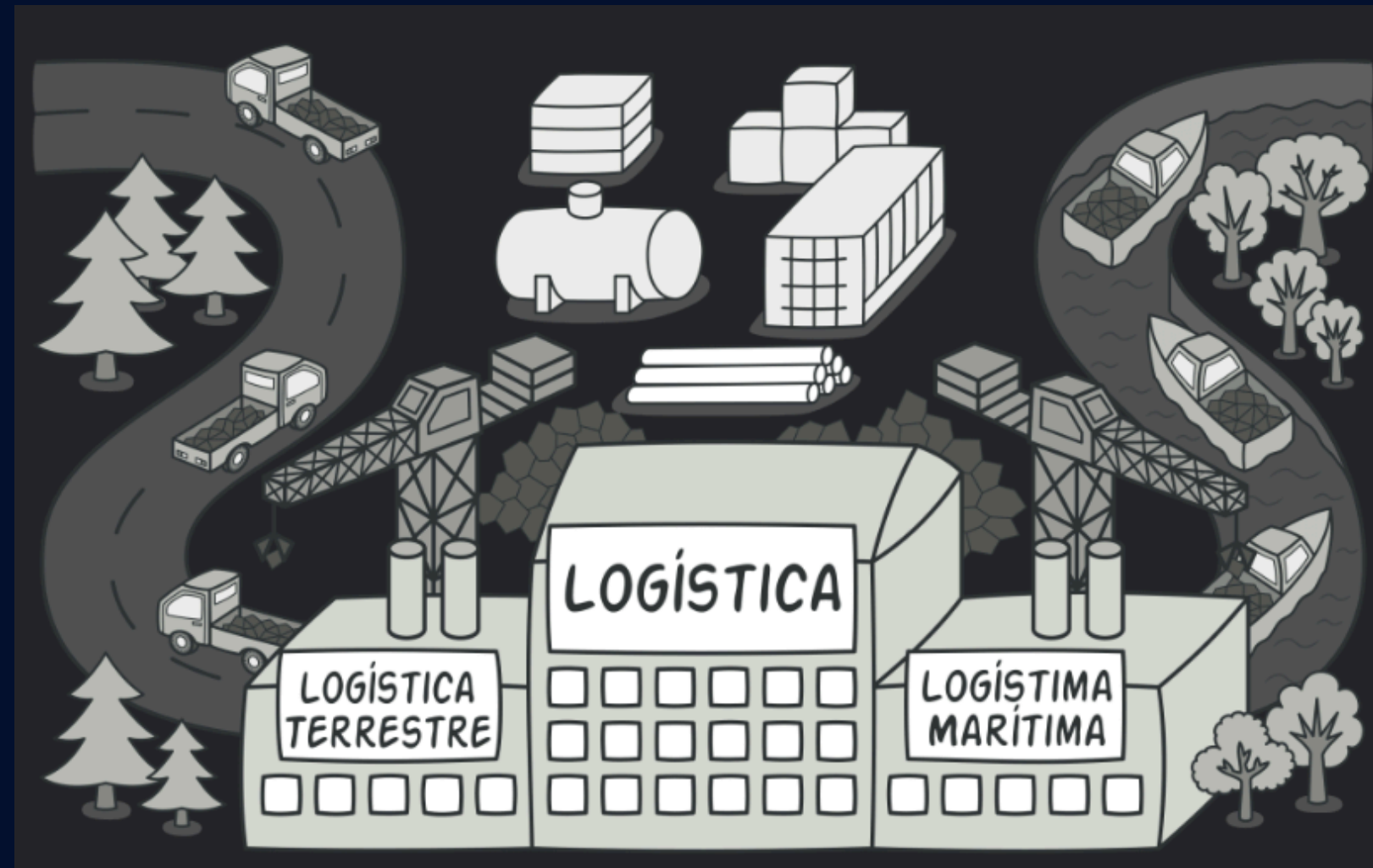
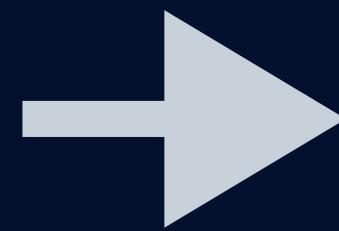


Ilustración 1: Refactoring Guru

Problema

Imagina que estás creando una aplicación de gestión logística. La primera versión de tu aplicación sólo es capaz de manejar el transporte en camión, por lo que la mayor parte de tu código se encuentra dentro de la clase `Camión`.



Al cabo de un tiempo, tu aplicación se vuelve bastante popular. Cada día recibes decenas de peticiones de empresas de transporte marítimo para que incorpores la logística por mar a la aplicación.



Ilustración 2: Refactoring Guru

¿QUE PASA CON EL CÓDIGO?

La mayor parte del código está acoplado a la clase `Camión`.

Para añadir barcos a la aplicación habría que hacer cambios en toda la base del código. Además, si más tarde se decide añadir otro tipo de transporte a la aplicación, probablemente tendrás que volver a hacer todos estos cambios.

Al final se termina con un código bastante sucio, plagado de condicionales que cambian el comportamiento de la aplicación.



SOLUCIÓN

El patrón Factory Method sugiere que, en lugar de llamar al operador `new` para construir objetos directamente, se invoque a un método fábrica especial. Así los objetos se siguen creando a través del operador `new`, pero se invocan desde el método fábrica. Los objetos devueltos por el método fábrica a menudo se denominan productos.

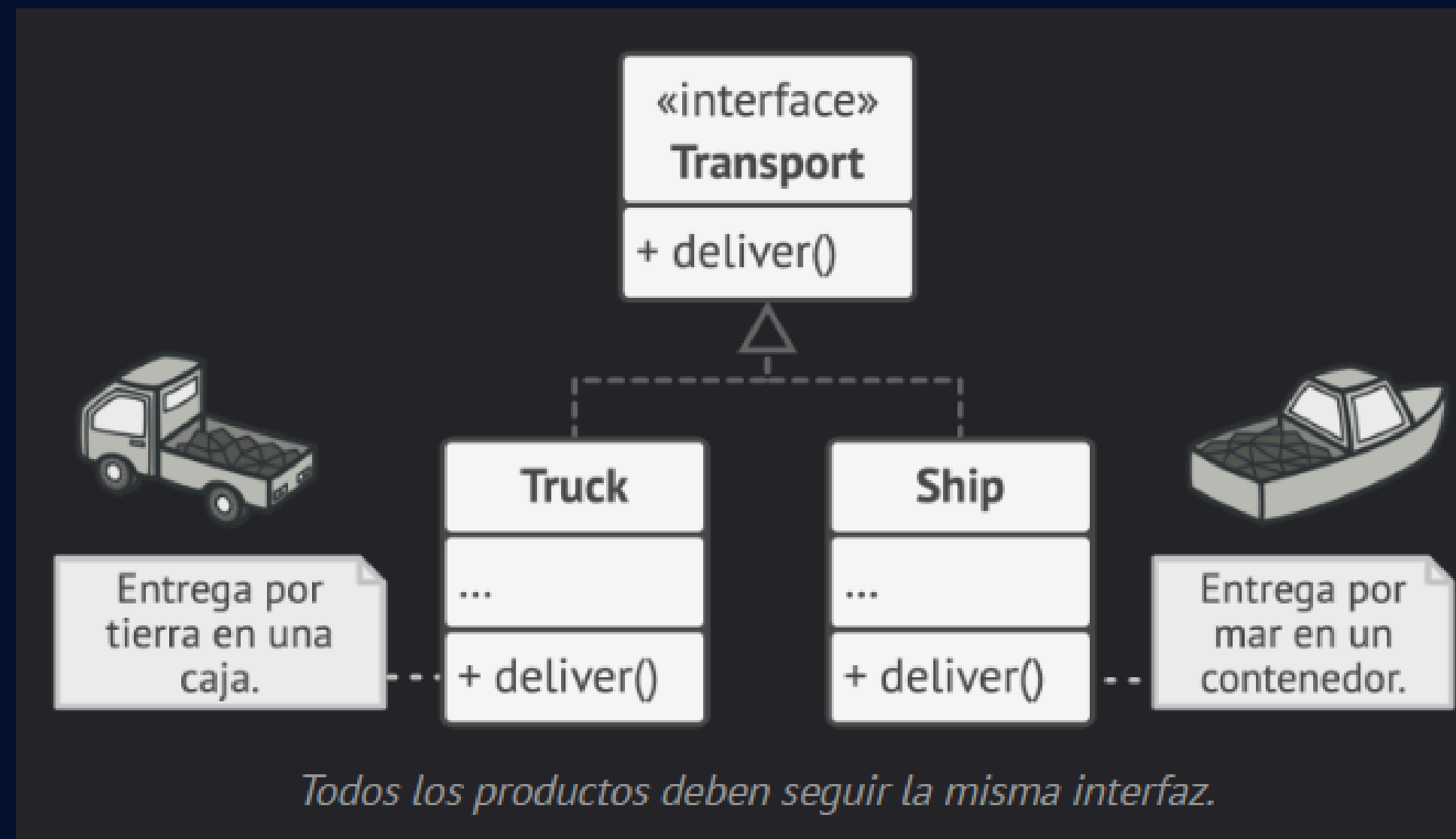


Ilustración 3: Refactoring Guru

IMPLEMENTACIÓN



Haz que todos los productos sigan la misma interfaz, la cual declare métodos que tengan sentido en todos los productos



Añade un patrón Factory Method vacío dentro de la clase creadora.



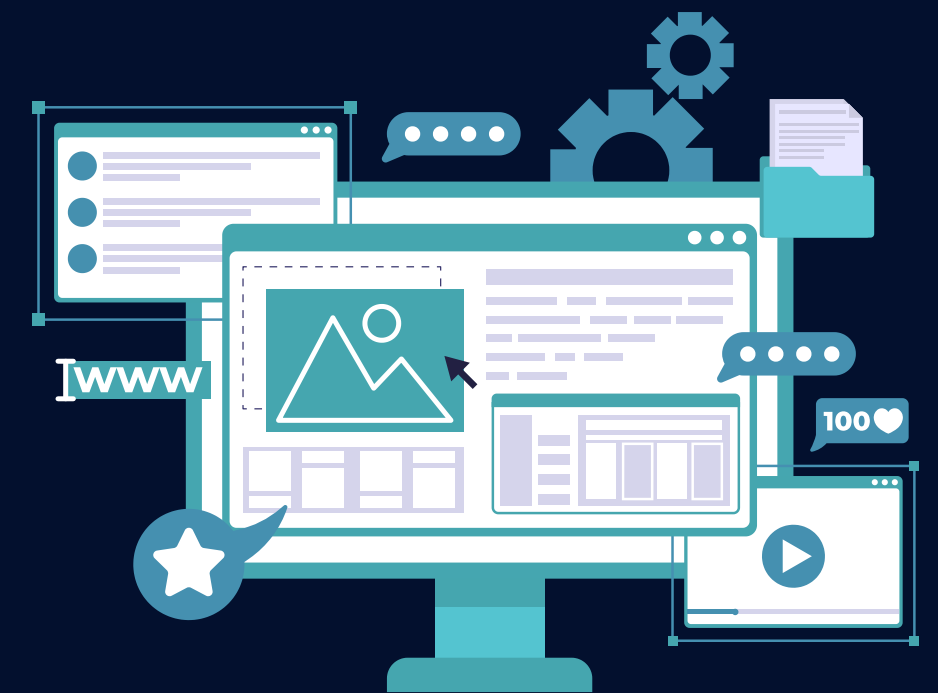
Encuentra todas las referencias a constructores de producto en el código de la clase creadora.



Ahora, crea un grupo de subclases creadoras para cada tipo de producto enumerado en el Factory Method.



Si hay demasiados tipos de producto y no tiene sentido crear subclases para todos ellos, puedes reutilizar el parámetro de control de la clase base en las subclases.



PROS



- Evitas un acoplamiento fuerte entre el creador y los productos concretos.
- Principio de responsabilidad única.
- Principio de abierto/cerrado.

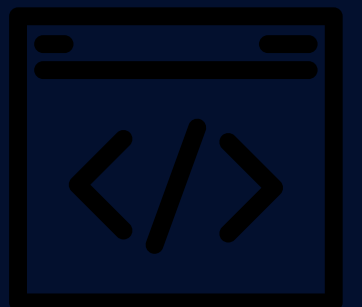
CONTRAS



- Puede ser que el código se complique, ya que debes incorporar una multitud de nuevas subclases para implementar el patrón. La situación ideal sería introducir el patrón en una jerarquía existente de clases creadoras.

RELACIÓN CON OTROS PATRONES

- Muchos diseños empiezan utilizando el Factory Method (menos complicado y más personalizable mediante las subclases) y evolucionan hacia Abstract Factory, Prototype, o Builder (más flexibles, pero más complicados).
- Las clases del Abstract Factory a menudo se basan en un grupo de métodos de fábrica, pero también puedes utilizar Prototype para escribir los métodos de estas clases.
- Puedes utilizar el patrón **Factory Method** junto con el **Iterator** para permitir que las subclases de la colección devuelvan distintos tipos de iteradores que sean compatibles con las colecciones.
- **Factory Method** es una especialización del **Template Method**. Al mismo tiempo, un Factory Method puede servir como paso de un gran Template Method.



EJEMPLO DE CÓDIGO



Ilustración 4: Refactoring Guru

Referencias.

- Refactoring.Guru. (s.f). Factory Method. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/factory-method>
- Refactoring.Guru. (s.f). Propósito de Factory Method [Ilustración 1]. En Factory Method. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/factory-method>
- Refactoring.Guru. (s.f). Problema de añadir nueva clase al programa [Ilustración 2]. En Factory Method. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/factory-method>
- Refactoring.Guru. (s.f). Solución con el patrón de Factory Method [Ilustración 3]. En Factory Method. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/factory-method>
- Refactoring.Guru. (s.f). Ejemplos de código [Ilustración 4]. En Factory Method. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/factory-method>

Muchas gracias.

